BCA309T: Cryptography & Network Security

ACADEMIC SOLUTIONS MANUAL

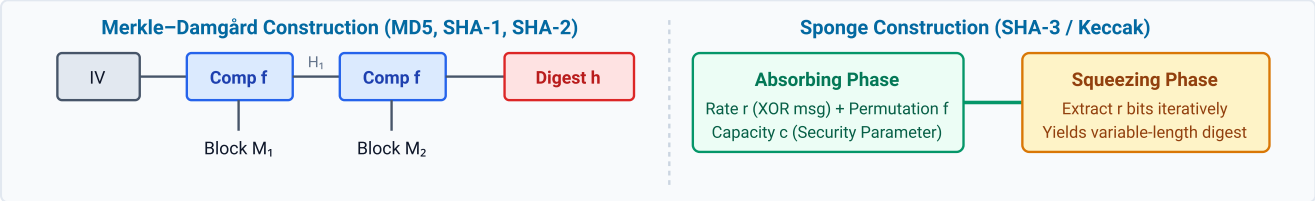
Assignment 5: Introduction to Hash Cryptography (BCA309T-AS06)

Q1 Definition and Fundamental Principles of Cryptographic Hash Functions

Hash Cryptography is the study and application of unkeyed mathematical algorithms that map arbitrary-length binary inputs $M \in \{0, 1\}^*$ into a strictly fixed-length, deterministic output digest $h \in \{0, 1\}^n$: $h = H(M)$. Unlike symmetric or asymmetric encryption, hashing is strictly **one-way** and **irreversible**—there is no secret key to recover the plaintext.

Core Architectural Principles:

- **Determinism:** The identical input message M will invariably yield the exact same bit digest h.
- **Fixed-Length Output:** Whether the input is a single character or a 5 TB database, the output length is constant (e.g., 256 bits for SHA-256).
- **Computational Efficiency:** Computing H(M) is lightweight and linear in time complexity O(|M|).
- **Avalanche Effect:** Inverting a single bit in the message changes approximately 50% of the digest bits unpredictably.



Q2 Role of Hash Functions in Cryptographic Applications

- **Data Integrity Verification:** Allows recipients of downloaded software or files to verify that no accidental bit corruption or malicious tampering occurred by matching computed hashes against published digests.
- **Digital Signatures:** Public-key encryption of bulk data is computationally prohibitive. Senders hash the payload to a 256-bit digest and encrypt only the digest with their private key: S = \text{Sign}_{PR}(H(M)).
- **Secure Password Storage:** Systems never store plaintext credentials. Passwords are concatenated with a cryptographically random *salt* and passed through adaptive hash/KDFs (e.g., bcrypt, Argon2, PBKDF2), neutralizing rainbow table attacks.
- **Message Authentication Codes (HMAC):** Incorporates a shared symmetric secret key K into the hash process: \text{HMAC}(K, M) = H((K^+ \oplus \text{opad}) \parallel H((K^+ \oplus \text{ipad}) \parallel M)), ensuring both message integrity and authentic origin.

Q3 Security Properties of Hash Functions & The Birthday Attack

- **Preimage Resistance (One-Way):** Computationally infeasible to find M given digest h such that H(M) = h. Requires 2^n operations.
- **Second-Preimage Resistance (Weak Collision):** Given input M_1, infeasible to find $M_2 \neq M_1$ such that $H(M_1) = H(M_2)$. Work: 2^n.
- **Collision Resistance (Strong Collision):** Infeasible to find *any* arbitrary pair $M_1 \neq M_2$ with $H(M_1) = H(M_2)$. Due to the **Birthday Paradox**, this requires only $2^{n/2}$ operations, halving effective bit security (e.g., SHA-256 provides 128-bit collision security).

Q4 The Secure Hash Algorithm (SHA) Family: SHA-1, SHA-2, and SHA-3

Feature / Metric	SHA-1	SHA-2 (SHA-256 / SHA-512)	SHA-3 (Keccak)
Standardized Year	1995 (FIPS PUB 180-1)	2001 (FIPS PUB 180-2)	2015 (FIPS PUB 202)
Output Digest Size	160 bits	224, 256, 384, 512 bits	224, 256, 384, 512 bits (and SHAKE XOF)
Internal Architecture	Merkle–Damgård	Merkle–Damgård (Davies–Meyer core)	Sponge Construction (Keccak-f[1600])
Rounds & Word Size	80 rounds; 32-bit words	64 rounds (256-bit) / 80 rounds (512-bit)	24 rounds on 5×5×64-bit state array
Security Status	**Broken** (Practical collisions: SHAttered 2017)	**Secure** (Industry baseline)	**Highly Secure** (Immune to length extension)

Q5 Applications of SHA in Network Protocols and Information Security

- **X.509 Digital Certificates:** Certificate Authorities (CAs) compute the SHA-256 digest of identity details and domains, appending their RSA/ECDSA signature to validate TLS server authenticity.
- **Digital Signatures & Code Authenticity:** Software binaries (e.g., Windows Authenticode, Android APKs) embed SHA-256 digests signed by software publishers to guarantee software has not been altered with trojans or spyware.
- **File & Blockchain Integrity:** Distributed systems like Git use SHA hashes to index trees and commits. Cryptographic blockchains (e.g., Bitcoin) utilize double-SHA256 (H(H(M))) for Proof-of-Work consensus.
- **Secure Communication Protocols (TLS 1.3 & IPsec):** TLS leverages HKDF (HMAC-based Key Derivation using SHA-256/384) to compute session encryption keys, maintaining a running transcript hash to verify handshake integrity.